

Memorie

Utilizziamo memorie a semiconduttore per

- I. Basso costo
 - II. Semplicità operativa (pochi transistor, celle semplici)
 - III. Dimensioni ridotte 7nm per cella
 - IV. Alte capacità
 - ↳ efficienza in area di memoria (bit per unità di area)
 - ↳ velocità di accesso ai dati (tempo di accesso)
 - ↳ potenza dissipata (importante di recente)
- densità bit-chip è raddoppiata ogni 2 anni (legge di Moore)
- 1980 → 100 $\frac{\text{kbit}}{\text{chip}}$ fino a 2010 → 100 $\frac{\text{Gbit}}{\text{chip}}$

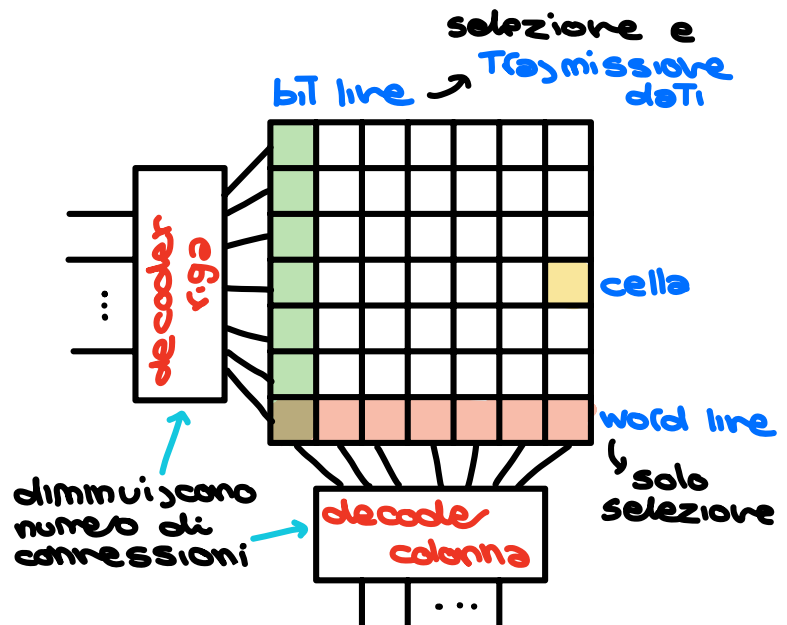
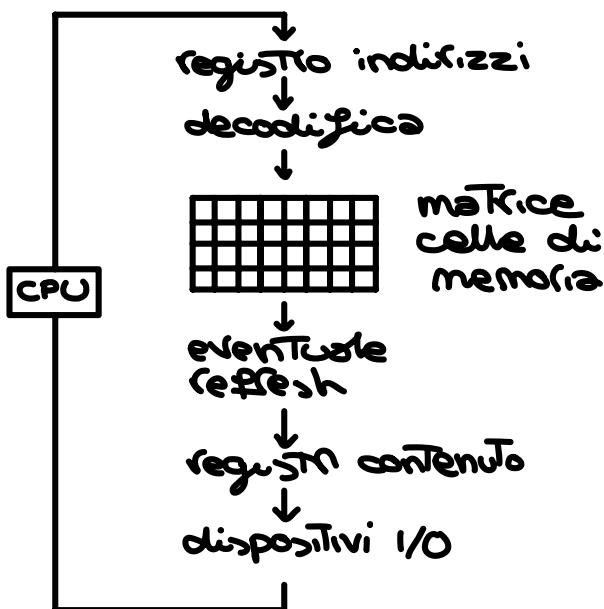
Tecnologia è estremamente complessa, vediamo le basi
Si dividono in

Memorie statiche: vanno sempre alimentate (l'info. è codificata nel passaggio di corrente)

Memorie dinamiche: alimentate ogni 2-5ms (info. codificata in accumuli di carica)

↳ più costose e più difficili da mantenere

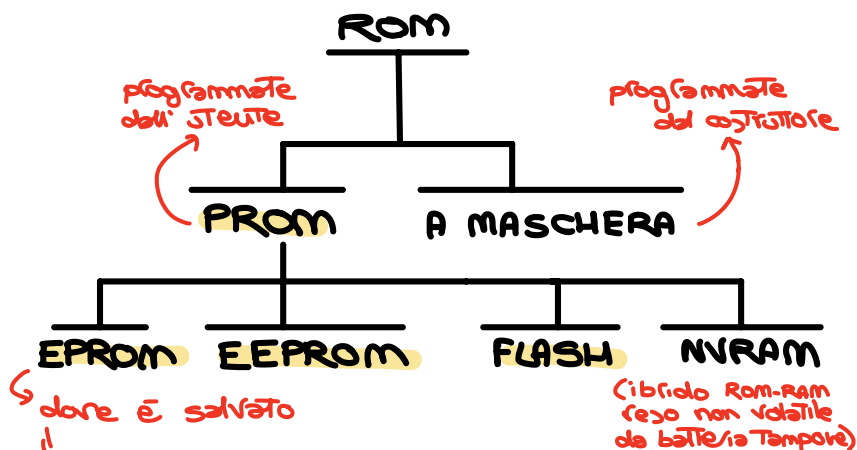
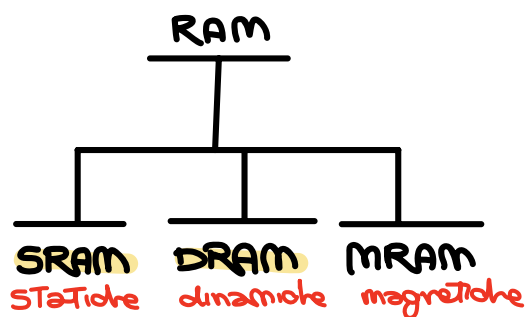
In ogni caso le memorie sono **circuiti bistabili** a 2 stati (1,0)
per estrarre il dato la memoria deve essere alimentata



Altra divisione delle memorie è

- I. ROM (Read-Only-Memory); solo lettura dati
 - ↳ non-volatili; mantengono dato anche se spenti
- II RAM (Random-Access-Memory); lettura + scrittura
 - ↳ volatili; perdono dato se spenti
- III Sequenziali; lettura solo nell'ordine di inserimento originale
 - ↳ Tempi di accesso maggiori
 - ↳ dischi, nastri magnetici, CERN → 1 TB di dati ogni 25

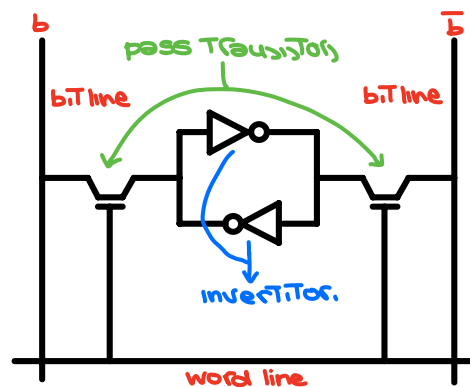
- ↳ utili se ho grandi quantità di dati che arrivano più veloci di come riesco a gestirli
- ↳ immagazzina tutto e sposta fuori alla sua velocità
- ↳ può avere **finito** in ingresso (**ATLAS** → solo KHz)



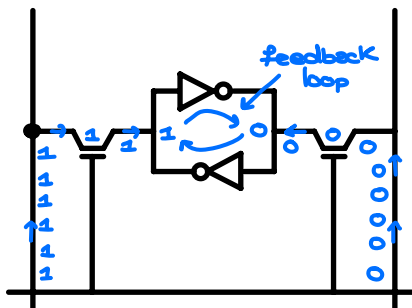
SRAM

Vediamo cella a 6 MOS (2 latch + 2+2 invertitori.)

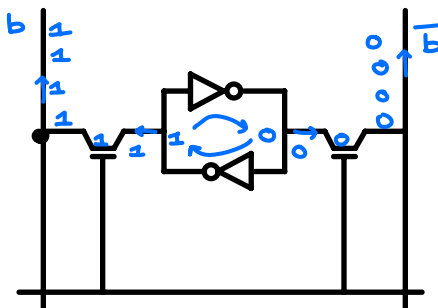
- la cella viene selezionata da WL e BL
- Praticamente il dato "gira" tra i due invertitori finché non viene letta la cella
- Ci sono due bitline perché anziché leggere b si legge $b-\bar{b}$ e si vede se da 0,5 → 1/0



scrittura (caso 1)



lettura (caso 1)



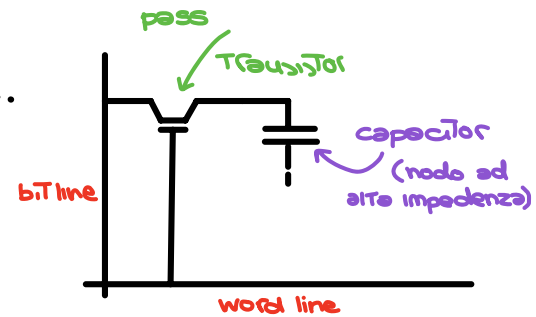
mettiamo 2 bit line per contrastare effetti parassiti e invertitori

la bitline è lunga e gli invertitori deboli, per quello guardiamo $b-\bar{b}$, non riuscirebbero a portare tutta la BL a 1 o 0. Quindi mettiamo BL a 0,5 e vediamo dove si avvicina

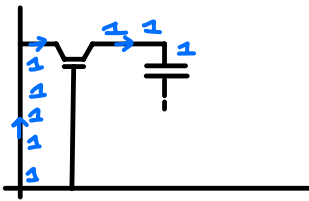
DRAM

Cella a 1 MOS

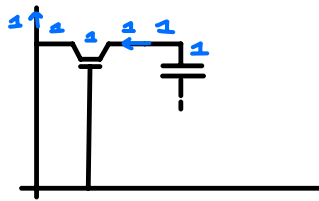
- Transistor latch collega il condensatore
- svuotiamo o riempiamo se vogliamo 0/1.
- Correnti parassite fanno svuotare carica anche se transistor chiuso
- In realtà cap. è pezzo di conduttore sepolto nel silicio della cella
 - ↳ per tenere più carica il capoc. deve essere grande



scrittura



lettura



con 1 si carica, con 0 si scarica

↳ correnti parassite
devo riscrivere spesso
(2-5 ms)

lascia carica lungo
bit line

↳ lettura distruttiva, devo riscrivere
ogni volta che leggo (refresh)

ma io voglio ridurre cella.

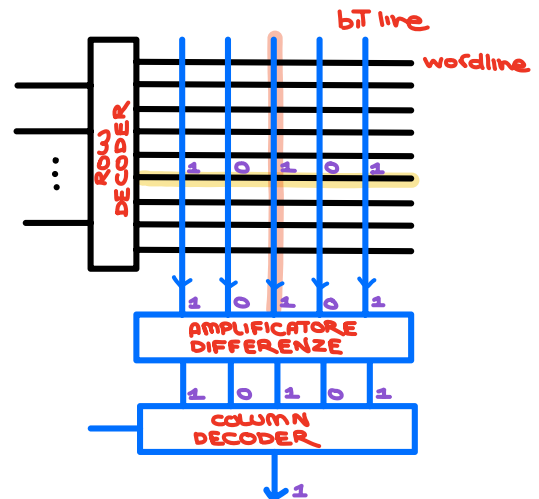
cella DRAM < cella SRAM
cella SRAM + velocità di DRAM

- La bit line è preventiv.
caricata per non far
svuotare cap. su di lei

DECODER

serve a selezionare celle

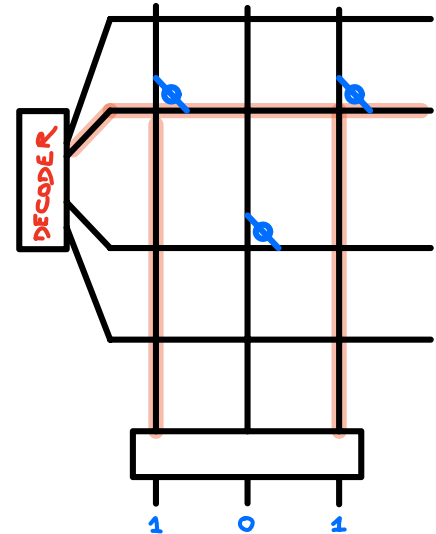
- In ingresso ho n bit di indirizzo
- In uscita 2^n bit della wordline
 - ↳ solo una attivata alla volta
 - ↳ a volte porta enable, serve a controllare se output è attivo
- L'amplific. differenziale sulle bit line riconosce il segnale debole delle celle e lo converte in 1 o 0.
- Il decoder di colonna riceve i bit e sceglie quali tenere in base a indirizzo.
- Nel caso di DRAM dopo riscrivo
- Esistono NOR/NAND decoder ma funzionamento uguale



ROM

Per le ROM ho solo lettura, l'informazione sta nel passaggio di corrente

- ↳ La selezione di una wordline fa passare corrente
- ↳ In fase di costruzione **non tutti** i gate sono stati messi.
- ↳ Passa corrente solo dove c'è il Transistor e lì avrà 0.



Una volta scelta la configurazione non è possibile cambiarla.

↳ Read-only

Se le voglio rendere programmabili diventano **PROM**

- ↳ Introduco tutti i gate ma **fusibili** (fusibile). Seleziono quali gate bruciare e applico tensione alta. Rimangono solo i gate che voglio.
- ↳ programmabile una volta sola.

EPROM

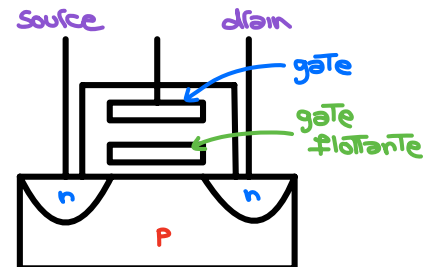
Erasable - Programmable - ROM

Aggiunge alle ROM possibilità di programmare e cancellare

- ↳ Si basa sul **MOS floating gate**

Ha 2 gate, uno collegato e uno sepolto nell'ossido

- ↳ Applicando **15-20V** (poco più dei **12-18V** al drain) si ha **iniezione a valanga** di elettroni nel gate sepolto



↳ e passano attraverso isolante per via di alte tensioni

Una volta carico il gate flottante schermo il gate normale, impedendo la formazione del canale → gate rimane spento. Posso programmare alle normali tensioni operative.

↳ gate rimane spento. Posso programmare alle normali tensioni operative.

- ↳ Si può cancellare illuminando con **luce UV**, che dà agli elettroni energia per uscire dal gate flottante.
- devo lasciare finestre per la luce sul chip (**quarzo**)
- ogni volta devo cancellare tutto (**~5/30min**) → perdo tempo
- per via della finestra hanno vita breve in cicli di prog.

253,7 nm

Quindi se V_{th} quella vecchia e V'_{th} quella con flottante pieno, operando con Tensione intermedia V_{GS} si ha

- a Transistor programmato, spento ($V_{GS} < V_{th}$)
- a Transistor cancellato, acceso ($V_{GS} > V'_{th}$)

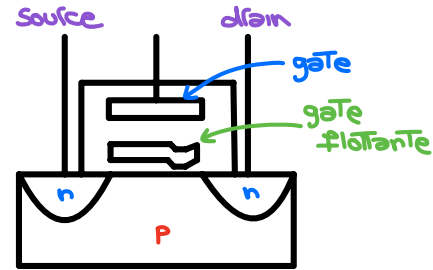
Allo stato programmato assegniamo valore 1 in quanto non scarica sulla bitline, e quello non programmato 0 perché è transistor uolante.

EEPROM

Electrically-Erasable-Programmable ROM
Si cancellano elettricamente

Funzionano con FETs modificati (FLOTOX)

- C'è una zona di separazione sottile tra gate flottante e canale (10nm di ossido)
- Viene applicata Tensione positiva al gate e gli elettroni migrano nel FG per effetto Tunnel. Applicando Tensione negativa tornano nel canale (cancellazione)
- La cancellazione è molto rapida (0,1-5 ms)



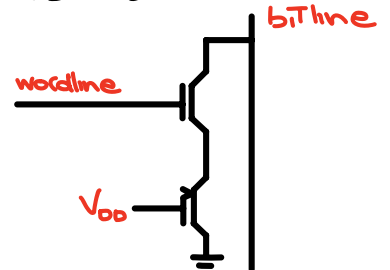
Tuttavia

- Per cancellare applico Tensione negativa al gate
- Serve Transistor di accesso per impedire accumuli nel FG di carica positiva
- Quindi la cella occupa di più, servono 2 Transistor
- I Transistor di accesso si possono "riciclare" per porre Tensioni da soglia basse per lo 0 e $> V_{DD}$ per il 1.
- In questo modo sarà sempre acceso se 0 e sempre spento se 1

In questo modo il FLOTOX è collegato alla WL solo se il Transistor di accesso è aperto, evitando Tunnel parziali.

Le limitazioni sono

- Minore densità di memoria x celle più grandi
- Durante riscritture l'ossido riceve elettroni quindi gradualmente scherma il FG
- Ad un certo punto 1 e 0 saranno indistinguibili perché FG e ossido si schermano (retenzione)
- Durante immagazzinamento elettroni del FG possono andare nell'ossido e perdere programmazione (ritenzione)
- Lo strato di ossido sottile è difficile e costoso
- Per leggere, programmare, cancellare servono V diversi



→ 0,1-5 ms per byte

FLASH

È un FLOTOX con tutto il gate flottante soggetto a effetto Tunnel

FLASH-NOR: $\left\{ \begin{array}{l} \text{drain sulla bitline} \\ \text{gate sulla wordline} \\ \text{source collegato a common source} \end{array} \right.$

- ↳ Programmazione avviene tramite iniezione elettroni caldi
 - Applico $V_{GS} > V_{DS}$ e fa iniezione
 - Quindi è programmabile bit a bit perché posso applicare le tensioni ad un solo FLASH

Per il resto come le EEPROM

→ 10 μ s-1ms per gruppo celle

- ↳ Cancellazione tramite effetto Tunnel sul source
 - Avviene a blocchi perché invertiamo V_{GS} quindi il common source di tutto il blocco viene abbassato.
 - ↳ le celle di un blocco condividono lo stesso substrato, non sono tutte isolate. Quindi se applico un campo tutto il substrato lo sente
- Tuttavia dopo $10^4/10^5$ cicli deteriorano.

Esistono anche le FLASH-NAND dove le celle sono in serie e le operazioni avvengono a blocchi

- ↳ Più veloci ma bisogna fare tutto a blocchi
- ↳ La programmazione avviene per effetto Tunnel

TABELLA GRANDEZZE

Refresh memorie dinamiche: 2-5 ms

	dim. celle	t cancellaz
ROM		X
PROM		X
EPROM		5-30 min
E ² PROM		0,1-5 ms
FLASH		0,01-1 ms
SRAM	6T	
DRAM	1T	

Cicli di scrittura flash; 10^6

Capacità cella DRAM; 50 fF capacità parassite; 1-5 fF

Tempi EEPROM; 1-10ms scrittura, 10-15 ns lettura

Tempi lettura-scrittura DRAM; 10-15 ns

SRAM; 0,5-5 ns